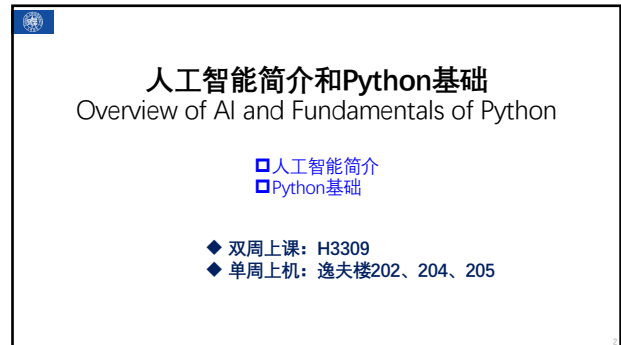
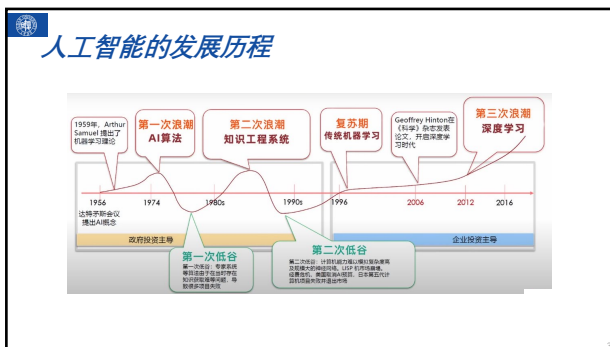




1



2



3



4



5



6

## Hello World程序

- 打印Hello World
  - 注释: 每一行 # 后面都是注释
  - Python函数: `print`
  - 输入: 字符串 "Hello World"
  - 输出: "Hello World"

```
#打印Hello World
print("Hello World")
[1] ✓ 0.0s
... Hello World
```

```
#打印Hello World到屏幕
print("Hello World")
#也可以用单引号: print('Hello World')
```

7

## Python程序运行

- `python hello.py`
- python命令
- Jupyter notebook
- 通过IDE进行运行和Debug
  - 适合编写规模较大的项目和工程

```
% python3 hello.py
Hello World

% python3
>>> print("Hello World")
Hello World
>>> quit()
```

```
#打印Hello World
print("Hello World")
[1] ✓ 0.0s
... Hello World
```

8

## 更多Python的例子

- `print(2 * 3)`
  - 2和3是常量, 不可以改变
- a、b、c是变量, =为赋值符号
  - `a = 2` #a赋值2
  - `b = 3` #b赋值3
  - `c = a + b` #先算a+b, 结果赋值c
  - `c = a * b` #先算a\*b, 结果赋值c

```
print(2*3)
[2] ✓ 0.0s
... 5

a = 2
b = 3
c = a + b
print(c)
c = a * b
print(c)
[3] ✓ 0.0s
... 5
... 6
```

9

## 温度转换程序

- 利用Python程序进行摄氏度和华氏度之间的转换
  - 转换公式如下:
  - $C = (F - 32) / 1.8$
  - $F = C * 1.8 + 32$

```
#TempConvert.py
TempStr= input("请输入带有符号的温度值: ")
if TempStr[-1] in ['F','f']:
    C = (float(TempStr[0:-1])-32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C','c']:
    F = float(TempStr[0:-1])*1.8+32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

```
% python3 TempConvert.py
请输入带有符号的温度值: 50F
转换后的温度是10.00C
```

10

## Python程序的格式框架

- Python语言采用严格的“缩进”来表明程序的格式框架。
  - 标明代码的层次关系, 可以多层
- Python的单行注释以#号开头
  - 多行注释以''' 开头和结尾(做大地说明)

```
'''
多行注释
'''
#单行注释

#TempConvert.py
TempStr= input("请输入带有符号的温度值: ")
if TempStr[-1] in ['F','f']:
    C = (float(TempStr[0:-1])-32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C','c']:
    F = float(TempStr[0:-1])*1.8+32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

11

## 标识符

- 常量和变量
  - 常量: 程序中值不发生改变的元素
  - 变量: 程序中值发生改变或者可以发生改变的元素
- Python语言的标识符
  - 字母、数字、下划线、汉字组合, 但首字符不能是数字
  - 长度无限制, 大小写敏感, 不能使用保留字

```
#TempConvert.py
TempStr= input("请输入带有符号的温度值: ")
if TempStr[-1] in ['F','f']:
    C = (float(TempStr[0:-1])-32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C','c']:
    F = float(TempStr[0:-1])*1.8+32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

|        |        |         |          |          |        |       |
|--------|--------|---------|----------|----------|--------|-------|
| False  | None   | True    | and      | as       | assert | async |
| await  | break  | class   | continue | def      | del    | elif  |
| else   | except | finally | for      | from     | global | if    |
| import | in     | is      | lambda   | nonlocal | not    | or    |
| pass   | raise  | return  | try      | while    | with   | yield |

12

## 输出- print函数

- 基本用法
 

```
print("Hello, World!")
```
- 格式化输出
 

```
name, age, height = "Alice", 25, 1.6234
print(f"我的名字是 {name}, 我今年 {age} 岁, 身高 {height:.2f} 米。")
```

13

## 输入- input函数

- 基本用法
 

```
user_input = input("请输入一些文本: ")
```
- 输入类型转换
 

```
age = int(input("请输入您的年龄: ")) # 获取整数输入
height = float(input("请输入您的身高 (米): ")) # 获取浮点数输入
```
- 处理多个输入
 

```
# 假设用户输入两个数字, 用空格分开
num1, num2 = map(int, input("请输入两个整数, 用空格分隔: ").split())
```
- 异常处理
 

```
try:
    age = int(input("请输入您的年龄: "))
except ValueError:
    print("无效输入, 请输入一个整数!")
```

■ input()是一个系统内置的函数,

> <变量> = input(<提示性文字>)

> 功能: 获取并返回用户在命令行中的输入字符串

14

## 同步赋值

- 同步赋值语句:同时给多个变量赋值
  - > <变量1>, ..., <变量N> = <表达式1>, ..., <表达式N>
  - > A, B = 12, 13 等价于 A = 12 B = 13
  - > 好处:省代码
- 举例:交换A和B的值

```
A = 10
B = 20
print(A,B)
A, B = B, A
print(A,B)
```

```
10 20
20 10
```

15

## 字符串

- Python语言中的字符串
  - > 两个双引号 "字符串"
  - > 或者单引号 '字符串'
- Python字符串的两种序号体系
  - > 从左到右, 从0开始逐位递增
  - > 从右到左, 从-1开始, 逐位递减
- 示例中的字符串处理
  - > TempStr[-1]:取字符串最后一个字符
  - > Temp[0:-1]:取字符串从第1个到倒数第二个

```
#TempConvert.py
TempStr= input("请输入带有符号的温度值: ")
if TempStr[-1] in ['F','f']:
    C = (float(TempStr[0:-1])-32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C','c']:
    F = float(TempStr[0:-1])*1.8+32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

反向递增序号

|     |     |    |    |    |    |    |    |    |    |    |
|-----|-----|----|----|----|----|----|----|----|----|----|
| -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |
| H   | e   | i  | l  | o  | W  | o  | r  | i  | d  |    |
| 0   | 1   | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |

正向递增序号

```
TempStr= input("请输入带有符号的温度值: ")
```

表示: 把input(用户输入的字符串) 返回的值放到TempStr这个字符串变量中

16

## 流程控制语句-1

```
#TempConvert.py
TempStr= input("请输入带有符号的温度值: ")
if TempStr[-1] in ['F','f']:
    C = (float(TempStr[0:-1])-32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C','c']:
    F = float(TempStr[0:-1])*1.8+32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

- 条件语句
  - 1.if - 如果条件为真, 则执行其下的代码块。
  - 2.elif (else if) - 前一个条件不满足时, 检查下一个条件。
  - 3.else - 所有前面的条件都不满足时执行。

17

## 流程控制语句-2

- 循环语句
  - 4.while - 在给定条件为真时重复执行代码块。
  - 5.for - 用于遍历序列中的每个元素。
  - 6.range() - 结合 for 循环生成数字序列进行循环。
    - > range(stop)
    - > range(start,stop[,step])
    - > 返回一个range对象, 可产生一系列的整数, 从start (缺省为0) 开始, 直到stop为止 (不包括), step为步长, 缺省为1
  - 跳转语句
    - 7.break - 跳出当前循环。
    - 8.continue - 跳过本次循环
    - 9.pass - 相当于C语言中的空语句。

```
i=0
while i < 5:
    print(i)
    i+=1
```

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

```
for i in range(5):
    print(i)
```

```
for i in range(0,5):
    print(i)
```

```
for i in range(0,5,1):
    print(i)
```

18

## 流程控制语句-3

### try...except...finally 异常处理

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("除零错误:", e)
finally:
    print("无论是否发生异常, 都会执行这里的代码")
```

### with语句与文件操作

```
with open('file.txt', 'r') as file:
    content = file.read()
    print(content)
# 文件在这里自动关闭
```

19

## 模块

```
import numpy as np
from math import *
from math import sqrt as qt
from random import random
```

### 模块导入:

#### > import module [as alias]

- 如果该module首次导入, 寻找模块源文件, 加载模块, 创建模块对象
- module或者alias与该模块对应的模块对象绑定

#### > from module import obj [as alias]

- 从模块中导入特定的对象, 即obj与对象绑定
- from module import \*

### 模块的 \_\_name\_\_ 属性:

- > 作为script运行时, \_\_name\_\_ 等于 '\_\_main\_\_'
- > 否则被import时, \_\_name\_\_ 为模块的名字

20

## 函数定义和调用

### 全局变量和局部变量

```
def greet(name, age=16, gender="girl"):
    print(f"Hello, {name}, {age} years old {gender}!")
    #return name, age

greet("Alice") # 输出: Hello, Alice, 16 years old girl!
greet("Marry", 18) # 输出: Hello, Marry, 18 years old girl!
greet("Bob", gender="boy") # Hello, Bob, 16 years old boy!
```

### 函数定义

- > 函数定义使用 **def 关键字**, 后跟函数名和圆括号 ()。
- > 参数都应该放在这些圆括号内。
- > 还可以选择性地定义参数的**默认值**。
- > 函数体从下一行开始, 并且必须**缩进**。

### 函数类型

- > 无参数函数/带参数函数/关键字参数函数/默认参数值函数/不定数量位置参数函数 (\*args)/不定数量关键字参数函数 (\*\*kwargs)/返回值函数

21

## 基本数据类型

### ■ 整数: 没有大小限制

- > 十进制: 5, -5, 2\*\*200
- > 二进制: 0b11, -0b101
- > 八进制: 0o12
- > 十六进制: -0x12

```
a,b,c=5,-5,2**200
print(f"a={a},b={b},c={c}")
a,b,c,d = 0b11, -0b101, 0o12, -0x12
print(f"a={a},b={b},c={c},d={d}")
```

```
a=5,b=-5,c=1606938044258990275541962092341162602522202993782792835301376
a=3,b=-5,c=10,d=-18
```

### ■ 浮点数: 64位双精度浮点数

- > 小数: 3.14, 77., -.12
- > 科学计算: 96e4, 4.3e-3, 9.6E-5

```
print(f"({3.14}, {77.}, {-12})")
print(f"({96e4}, {4.3e-3}, {9.6E-5})")
```

### ■ 复数

- > 2+3j, 1-2j

```
3.14, 77.0, -0.12
960000.0 0.0043 9.6e-05
```

22

## 数字类型的基本运算

| 操作符/函数       | 描述                         | 例子                                                  |
|--------------|----------------------------|-----------------------------------------------------|
| +            | 加法: 两个数值相加。                | print(5 + 3) 输出: 8                                  |
| -            | 减法: 从一个数值中减去另一个数值。         | print(5 - 3) 输出: 2                                  |
| *            | 乘法: 两个数值相乘。                | print(5 * 3) 输出: 15                                 |
| /            | 除法: 一个数值除以另一个数值。结果总是浮点数。   | print(6 / 3) 输出: 2.0                                |
| //           | 整除: 返回除法的整数部分 (向下取整)。      | print(7 // 3) 输出: 2                                 |
| %            | 取模: 返回除法的余数。               | print(7 % 3) 输出: 1                                  |
| **           | 幂运算: 计算一个数值的指数幂。           | print(2 ** 3) 输出: 8                                 |
| abs()        | 绝对值: 获取数值的绝对值。             | print(abs(-5)) 输出: 5                                |
| math.floor() | 向下取整: 将数值向下舍入到最接近的整数。      | print(math.floor(4.7)) 输出: 4                        |
| math.ceil()  | 向上取整: 将数值向上舍入到最接近的整数。      | print(math.ceil(4.2)) 输出: 5                         |
| round()      | 四舍五入: 根据数值的小数部分四舍五入到最近的整数。 | print(round(3.6)) 输出: 4;<br>print(round(3.4)) 输出: 3 |
| int()        | 类型转换: 将数值转换为整数, 会截断小数部分。   | a = 5.9; print(int(a)) 输出: 5                        |
| float()      | 类型转换: 将数值转换为浮点数。           | c = 7; print(float(c)) 输出: 7.0                      |

23

## 字符串的基本操作-1

| 操作符/函数   | 描述                               | 例子                                            |
|----------|----------------------------------|-----------------------------------------------|
| +        | 字符串连接: 将两个字符串连接在一起。              | print("Hello" + ", World!") 输出: Hello, World! |
| *        | 字符串重复: 重复字符串若干次。                 | print("Hi-" * 3) 输出: Hi-Hi-Hi-                |
| in       | 检查一个子串是否在另一个字符串中。                | print('o' in 'Hello') 输出: True                |
| not in   | 检查一个子串是否不在另一个字符串中。               | print('z' not in 'Hello') 输出: True            |
| []       | 通过索引访问字符串中的特定字符。                 | s = "Hello"; print(s[1]) 输出: e                |
| []       | 切片: 提取字符串的一部分。                   | s = "Hello, World!"; print(s[0:5]) 输出: Hello  |
| .find()  | 查找子串: 返回子串首次出现的位置, 如果没有找到则返回 -1。 | print("Hello, World!".find("World")) 输出: 7    |
| .rfind() | 反向查找子串: 从右侧开始查找子串首次出现的位置。        | print("Hello, Hello".rfind("Hello")) 输出: 7    |

24

## 字符串的基本操作-2

| 操作符/函数                     | 描述                              | 例子                                                                                |
|----------------------------|---------------------------------|-----------------------------------------------------------------------------------|
| <code>.replace()</code>    | 替换子串：替换所有匹配的子串。                 | <code>print("Hello, World!".replace("World", "Python"))</code> 输出: Hello, Python! |
| <code>.upper()</code>      | 转大写：将字符串转换为全大写形式。               | <code>print("hello".upper())</code> 输出: HELLO                                     |
| <code>.lower()</code>      | 转小写：将字符串转换为全小写形式。               | <code>print("HELLO".lower())</code> 输出: hello                                     |
| <code>.title()</code>      | 标题化：将字符串中每个单词的首字母大写。            | <code>print("hello world".title())</code> 输出: Hello World                         |
| <code>.capitalize()</code> | 首字母大写：将字符串的第一个字符大写，其余转为小写。      | <code>print("hello".capitalize())</code> 输出: Hello                                |
| <code>.strip()</code>      | 去除空白：去除字符串两端的空白字符。              | <code>print(" hello ".strip())</code> 输出: hello                                   |
| <code>.lstrip()</code>     | 去除左端空白：去除字符串左端的空白字符。            | <code>print(" hello ".lstrip())</code> 输出: hello                                  |
| <code>.rstrip()</code>     | 去除右端空白：去除字符串右端的空白字符。            | <code>print(" hello ".rstrip())</code> 输出: hello                                  |
| <code>.split()</code>      | 分割字符串：按指定分隔符分割字符串，默认以空格作为分隔符。   | <code>print("Hello, World!".split(", "))</code> 输出: ['Hello', 'World!']           |
| <code>.join()</code>       | 连接序列：将序列中的元素以指定的字符串连接生成一个新的字符串。 | <code>print("-".join(["Hello", "World!"]))</code> 输出: Hello-World!                |

25

## 比较逻辑运算符

| 操作符                | 描述                      | 例子                                         |
|--------------------|-------------------------|--------------------------------------------|
| <code>==</code>    | 等于：检查两个值是否相等。           | <code>print(5 == 5)</code> 输出: True        |
| <code>!=</code>    | 不等于：检查两个值是否不相等。         | <code>print(5 != 3)</code> 输出: True        |
| <code>&gt;</code>  | 大于：检查左边的值是否大于右边的值。      | <code>print(5 &gt; 3)</code> 输出: True      |
| <code>&lt;</code>  | 小于：检查左边的值是否小于右边的值。      | <code>print(3 &lt; 5)</code> 输出: True      |
| <code>&gt;=</code> | 大于等于：检查左边的值是否大于等于右边的值。  | <code>print(5 &gt;= 5)</code> 输出: True     |
| <code>&lt;=</code> | 小于等于：检查左边的值是否小于等于右边的值。  | <code>print(3 &lt;= 5)</code> 输出: True     |
| <code>and</code>   | 逻辑与：如果两边的操作数都为真，则结果为真。  | <code>print(True and True)</code> 输出: True |
| <code>or</code>    | 逻辑或：如果任意一边的操作数为真，则结果为真。 | <code>print(True or False)</code> 输出: True |
| <code>not</code>   | 逻辑非：反转操作数的真假值。          | <code>print(not True)</code> 输出: False     |

26

## 文件写入

| 文件读取函数                           | 描述               | 例子                                                       |
|----------------------------------|------------------|----------------------------------------------------------|
| <code>open(filename, 'r')</code> | 以只读模式打开文件。       | with <code>open("example.txt", "r")</code> as file:      |
| <code>.read()</code>             | 读取整个文件内容。        | <code>content = file.read(); print(content)</code>       |
| <code>.readline()</code>         | 读取文件的一行。         | <code>line = file.readline(); print(line.strip())</code> |
| <code>.readlines()</code>        | 读取文件的所有行并返回一个列表。 | <code>lines = file.readlines(); print(lines)</code>      |
| for line in file                 | 遍历文件中的每一行。       | for line in file: <code>print(line.strip())</code>       |

| 文件写入函数                           | 描述               | 例子                                                                                   |
|----------------------------------|------------------|--------------------------------------------------------------------------------------|
| <code>open(filename, 'w')</code> | 以写入模式打开文件（覆盖模式）。 | with <code>open("example.txt", "w")</code> as file:                                  |
| <code>open(filename, 'a')</code> | 以追加模式打开文件。       | with <code>open("example.txt", "a")</code> as file:                                  |
| <code>.write()</code>            | 写入字符串到文件。        | <code>file.write("这是写入文件的第一行。\\n")</code>                                            |
| <code>.writelines()</code>       | 写入字符串列表到文件。      | <code>lines_to_write = ["新的一行\\n", "另一行\\n"]; file.writelines(lines_to_write)</code> |

27

## 组合数据类型

| 组合数据类型名        | 描述                    | 例子                                                  | 支持的基本操作                                                                                                                                                                                                                                                                                                                           |
|----------------|-----------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 列表(List)       | 有序的、可变的数据集合，支持异构元素。   | <code>my_list = [1, 2, "three", True]</code>        | <code>append()</code> , <code>extend()</code> , <code>insert()</code> , <code>remove()</code> , <code>pop()</code> , <code>index()</code> , <code>count()</code> , <code>sort()</code> , <code>reverse()</code>                                                                                                                   |
| 元组(Tuple)      | 有序的、不可变的数据集合。         | <code>my_tuple = (1, 2, "three")</code>             | <code>+</code> , <code>*</code> , <code>in</code> , <code>not in</code> , <code>len()</code>                                                                                                                                                                                                                                      |
| 字典(Dictionary) | 键值对形式的数据结构，键必须唯一且不可变。 | <code>my_dict = {"name": "Alice", "age": 30}</code> | <code>keys()</code> , <code>values()</code> , <code>items()</code> , <code>get()</code> , <code>update()</code> , <code>pop()</code> , <code>add()</code> , <code>remove()</code> , <code>discard()</code> , <code>union()</code> , <code>intersection()</code> , <code>difference()</code> , <code>symmetric_difference()</code> |
| 集合(Set)        | 无序且不重复的元素集合。          | <code>my_set = {1, 2, 3}</code>                     |                                                                                                                                                                                                                                                                                                                                   |

28

## 函数式编程

|                                                                                                                                                                                                                                                                                  |                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <pre>num1, num2 = map(int, input("请输入两个整数，用空格分隔：").split())</pre>                                                                                                                                                                                                                |                                   |
| <b>■ Lambda函数（匿名函数）</b><br><code>double = lambda x: x * 2</code><br><code>print(double(5))</code> # 输出: 10                                                                                                                                                                       |                                   |
| <b>■ 映射 (Map)、过滤 (Filter) 和归约 (Reduce)</b><br>➢ <code>map()</code> : 对列表中的每个元素应用函数并返回结果列表。<br>➢ <code>filter()</code> : 根据条件过滤列表中的元素。<br>➢ <code>reduce()</code> : 累积地应用一个二元函数来减少列表为单一值（需要从 <code>functools</code> 模块导入）。                                                        |                                   |
| <pre>from functools import reduce numbers = [1, 2, 3, 4]  squared = list(map(lambda x: x ** 2, numbers)) print(squared) filtered = list(filter(lambda x: x % 2 == 0, numbers)) print(filtered) product = reduce(lambda x, y: x if x &gt; y else y, numbers) print(product)</pre> | <pre>[1, 4, 9, 16] [2, 4] 4</pre> |

29

## 对象

- **对象：Python中各种数据的抽象**
- 标识`id(identity)`: 对象一旦创建其ID不再改变，可以看成该对象在内存中的地址。
  - `id(x)` 返回对象x的ID，`a is b` 判别 a和b是否同一个对象（ID是否相同）
- **类型(type)**: 决定了对象可能取值范围以及支持的操作。强类型语言
  - `type(x)` 返回对象x的类型
  - 对象是类型的一个实例（instance）
- **值(value)**: 值可能改变或者不可变，决定于其类型
  - 不可变(immutable): 有些对象一旦创建其值不可变，比如数字、字符串、元组等
  - 可变(mutable): 对象的值可以改变，比如列表、字典等

30

## 可变对象和不可变对象

### ■ 不可变对象

- int, float, bool, string, tuple等
- 不可变对象是指一旦创建后就不能修改其内容的对象。对不可变对象的任何“修改”实际上都是创建了一个新的对象，并让原来的变量名指向这个新对象。原始对象保持不变。

```
str1="Hello"
print(str1)
str1.upper()
print(str1) # Hello
str1 = str1.upper()
print(str1) # HELLO
```

### ■ 可变对象

- list, dict, set, numpy等
- 可变对象是指在其创建后可以修改其内容的对象。这意味着你可以在不改变对象身份（即内存地址）的情况下更改其值。如果需要独立的副本，则应使用拷贝方法

```
str1=list("Hello")
print(str1) # ['H', 'e', 'l', 'l', 'o']
str1[0]='m'
print(str1) # ['m', 'e', 'l', 'l', 'o']
```

31

## Python 面向对象编程

- Python 面向对象编程 (OOP) 是一种编程范式，它使用“类”和“对象”来组织代码。面向对象编程的核心概念包括封装、继承、多态和抽象。

- 类 (Class)：类是创建对象的蓝图或模板。定义对象的属性和方法。
- 对象 (Object)：对象是类的实例。包含具体的属性值和可以执行的方法。
- 封装 (Encapsulation)：将数据（属性）和操作数据的方法绑定在一起，并隐藏对象的内部实现细节。保护对象的状态，防止外部直接访问和修改。
- 继承 (Inheritance)：一个类可以从另一个类继承属性和方法。促进代码重用和层次化设计。
- 多态 (Polymorphism)：不同的类可以定义相同名称的方法，但行为不同。提高代码的灵活性和可扩展性。
- 抽象 (Abstraction)：隐藏复杂的实现细节，只暴露必要的接口。简化复杂系统的设计。

32

## 向量化

### ■ 向量化

```
c = np.dot(a,b)
```

### ■ 非向量化

```
for i in range(1000000):
    c += a[i]*b[i]
```

```
import numpy as np #导入numpy库
import time #导入时间库
a = np.random.rand(1000000)
b = np.random.rand(1000000)
#向量化的成本
tic = time.time()
c = np.dot(a,b)
toc = time.time()
print("%.9f"%c)
print("向量化方法运行时间:" + str(1000*(toc-tic)) + "ms")
#非向量化的成本
c = 0
tic = time.time()
for i in range(1000000):
    c += a[i]*b[i]
toc = time.time()
print("%.9f"%c)
print("For循环方法运行时间:" + str(1000*(toc-tic)) + "ms")

250180.043669234
向量化方法运行时间:3.6008358001708984ms
250180.043669226
For循环方法运行时间:237.39123344421387ms
```

33

## Numpy库

### ■ 什么是 NumPy?

- NumPy (Numerical Python) 是一个用于科学计算的强大库。它是 Python 生态系统中不可或缺的一部分，广泛应用于数据分析、机器学习、深度学习等领域。NumPy 提供了高性能的多维数组对象以及用于处理这些数组的工具。

### ■ 主要特点

- 多维数组对象 (ndarray)：支持高效存储和操作大型数组。数组中的元素类型相同，通常为数值类型（如整数、浮点数等）。
- 广播机制 (Broadcasting)：允许不同形状的数组进行算术运算，而无需显式复制数据。
- 丰富的数学函数：包含线性代数、傅里叶变换、随机数生成等功能。
- 高效的内存管理：数据在内存中连续存储，提高访问速度。
- 与其他库的兼容性：与 Pandas、Matplotlib、SciPy 等库无缝集成。

34

## 数组属性

### ■ 查看数组的形状

- shape = two\_dim\_array.shape

### ■ 查看数组的数据类型

- dtype = two\_dim\_array.dtype

### ■ 查看数组的维度

- ndim = two\_dim\_array.ndim

### ■ 查看数组的总元素数量

- size = two\_dim\_array.size

```
import numpy as np
# 创建一个一维数组
one_dim_array = np.array([1, 2, 3, 4, 5])
print("一维数组:", one_dim_array)
# 创建一个二维数组
two_dim_array = np.array([[1, 2, 3], [4, 5, 6]])
print("\n二维数组:\n", two_dim_array)

二维数组: [[1 2 3]
            [4 5 6]]

数组形状: (2, 3)

数据类型: int64

维度数量: 2

总元素数量: 6
```

35

## 数学运算

### ■ 加法

- add\_result = two\_dim\_array + 5

二维数组: [[1 2 3]  
[4 5 6]]

### ■ 乘法

- mul\_result = two\_dim\_array \* 2

加法结果: [[ 6 7 8] [ 9 10 11]]  
乘法结果: [[ 2 4 6] [ 8 10 12]]

### ■ 矩阵乘法

- matrix\_b = np.array([[9, 8], [7, 6], [5, 4]])
- dot\_product = np.dot(two\_dim\_array, matrix\_b)

矩阵乘积: [[38 32] [83 74]]

36

## 形状操作

### ■ 重塑数组

➢ `reshaped_array = one_dim_array.reshape(5, 1)`

➢

### ■ 转置矩阵

➢ `transposed_matrix = two_dim_array.T`

### ■ 展平数组

➢ `flattened_array = two_dim_array.flatten()`

一维数组: [1 2 3 4 5]

二维数组: [[1 2 3]  
[4 5 6]]

重塑后的数组: [[1] [2] [3] [4] [5]]

转置后的矩阵: [[1 4] [2 5] [3 6]]

展平后的数组: [1 2 3 4 5 6]

37

## 切片

### ■ 切片一维数组

➢ `sliced_one_dim = one_dim_array[1:4]`

一维数组: [1 2 3 4 5]

二维数组: [[1 2 3]  
[4 5 6]]

### ■ 切片二维数组

➢ `sliced_two_dim = two_dim_array[:, 1:]`

切片一维数组: [2 3 4]

切片二维数组: [[2 3]]

38

## 广播

广播 (Broadcasting) 是 NumPy 中一个非常强大且灵活的特性, 允许不同形状的数组进行算术运算, 而无需显式复制数据。广播规则使得数组操作更加高效和简洁。

### ■ 广播的基本规则

- 从后向前比较两个数组的维度。
- 如果两个维度大小相同, 或者其中一个维度大小为 1, 则这两个维度被认为是兼容的。

### ■ 扩展维度:

- 对于大小为 1 的维度, NumPy 会在内部将其扩展到与另一个数组相同的大小, 而不实际复制数据。

### ■ 不匹配的情况:

- 如果两个维度既不相同也不为 1, 则会引发 `ValueError`。

数组 a: [1 2 3]

数组 b:  
[[1]  
[2]  
[3]]

数组 a:  
[[1 2 3]  
[4 5 6]]

数组 b:  
[[0 1 2]]

结果:  
[[1 3 5]  
[4 6 8]]

数组 a:  
[[1 2 3]  
[4 5 6]]

数组 b:  
[[1 2]  
[3 4]]

`ValueError`:

39

## Matplotlib库

### ■ 什么是 Matplotlib ?

- Matplotlib 是一个用于创建静态、动态和交互式可视化的 Python 绘图库。它广泛应用于数据可视化领域, 支持多种图表类型, 如折线图、柱状图、散点图、直方图等。Matplotlib 是科学计算生态系统中的核心工具之一, 常与 NumPy、Pandas 和 SciPy 等库结合使用。

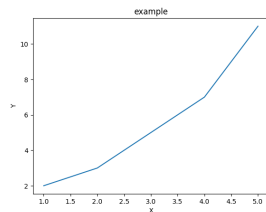
### ■ 主要特点

- 广泛的图表类型: 折线图、柱状图、散点图、饼图、直方图等。支持三维绘图。
- 高质量输出: 生成高分辨率的图像文件 (PNG、PDF、SVG 等)。支持 LaTeX 文本渲染, 适合学术出版物。
- 灵活性和定制性: 可以精细控制图表的每个细节, 如颜色、线条样式、标签、注释等。支持子图和多图布局。
- 集成性强: 与 Pandas、NumPy 等库无缝集成。支持 Jupyter Notebook 中的交互式绘图。
- 社区活跃: 拥有一个庞大且活跃的用户社区, 提供了丰富的文档和教程。

40

## Matplotlib库

```
import matplotlib.pyplot as plt
# 数据准备
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
# 绘制数据
plt.plot(x, y)
# 添加标题和轴标签
plt.title("example")
plt.xlabel("X")
plt.ylabel("Y")
# 显示图像
plt.show()
```

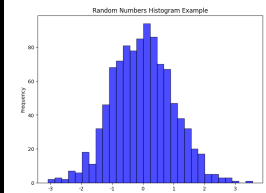


绘制折线图

41

## Matplotlib库

```
import numpy as np
import matplotlib.pyplot as plt
# 使用NumPy生成1000个符合标准正态分布(均值为0, 标准差为1)的随机数
data = np.random.randn(1000)
# 创建图形
plt.figure(figsize=(8, 6))
# 绘制直方图
plt.hist(data, bins=30, alpha=0.7, color='blue', edgecolor='black')
# 添加标题和轴标签
plt.title("Random Numbers Histogram Example")
plt.xlabel("Value")
plt.ylabel("Frequency")
# 显示图像
plt.show()
```



绘制直方图

42